Assignment 2

Introduction to Classes: Airspace Safety Determination in a Country

Introduction: In addition to its geographical boundary, each country has its own airspace. The airspace is determined by vertically drawing imaginary lines towards the sky up to a certain limit. The default boundary of a country's airspace is generally defined by the Kármán line, which is set at an altitude of 100 kilometers above sea level. Each country has its local coordinate system that can be used to determine the airspace height. Additionally, if an enemy plane enters the space, the position of the enemy plane is determined by that country's local coordinate system. The whole airspace can be roughly approximated as a large cylinder that has its center known as the origin of the country's local coordinate and the height that determines the airspace height. Enemy planes that enter the cylindrical airspace of a country can be detected by radars and readily destroyed.

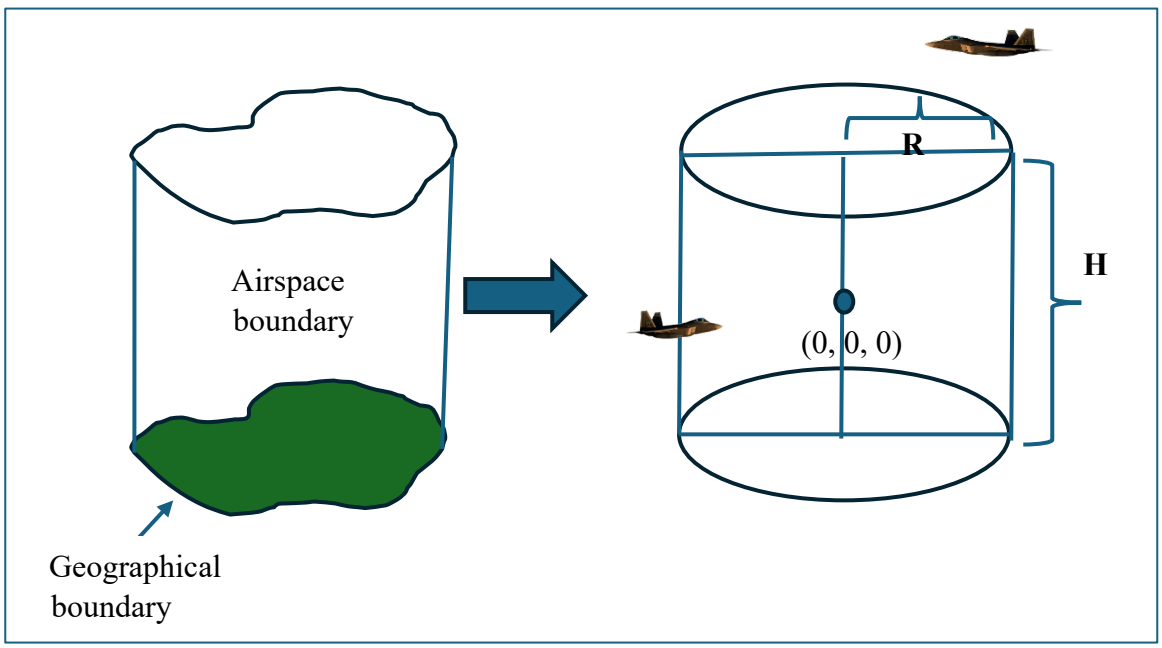

Figure 1: (Left) The geographical boundary (marked green) of a country and the airspace boundary. (Right) The original geographical boundary is approximated to a circular region, and the corresponding airspace can be approximated by a cylinder. An enemy plane enters the airspace boundary, and the other is outside of the boundary. A radar will detect the plane that enters the airspace boundary.

As part of the assignment, you will construct the cylindrical airspace boundary of a country from the center, radius, and height of a cylinder. Next, from the position of an enemy plane, the radar will determine if an enemy plane has entered its airspace boundary.

You will implement three classes: **Point3D**, **Airspace**, and **Radar**.

Assignment Specification:

Point3D Class: **3 points**

1) Constructors for the Point3D class:

A point can be defined in a 3D space by its x, y, and z components. The following expression defines a Point3D object **P** with its three member variables, **x**, **y**, and **z**. **x**, **y**, and **z** are of type **double**.

Point3D P{x, y, z}; ……..(i)

You need to implement the default constructor and constructor with parameters for Point3D class.

2) **Setters and getters**:
Add setters and getters to set the values of member variables and access them respectively.

3) **Add two Point3Ds**:
Write a public method that will add a Point3D to an existing point ('this' Point3D) and then returns the reference to it (this' Point3D). The function declaration will look like the following:

Point3D &addPoints(const Point3D & P); ……(ii)

4) **Subtract one Point3D from another Point3D:**
Write a public method that will subtract a Point3D from an existing point ('this' Point3D) and then return the reference to it. The function declaration will look like the following:

Point3D &subtractPoints(const Point3D & P); …… (iii)

5) **Magnitude of Point3D**:This method will find the magnitude of a Point3D object by squaring its X, Y, and Z components and returning the square root of it.

double magnitude() …… (iv)

Airspace Class: **4 points**

6) **Constructors for the Airspace class:**
As mentioned before, the airspace is a cylinder with its radius, height, and center. The center 'C' of the cylinder is a Point3D object; radius (R) and height (H) are of type double. Below is the constructor with parameter for the airspace class.

Airspace S{double R, double H, Point3D C}; ……..(v)

You need to implement the default constructor and constructor with parameters for Airspace class. **Setters and getters**: Add setters and getters to set the values of member variables and access them respectively.

Radar Class: **3 points**

7) **Constructors for the Radar class:**

A radar class will have the position of the enemy plane to determine whether the plane is inside or outside the airspace boundary. Hence, the member variable of the Radar class can be defined as a Point3D object that stores the position of a plane. Below is the constructor with parameter for the Radar class.

Radar rd{Point3D position};…….(vii)

You need to implement the default constructor and constructor with parameters for the Radar class. The default constructor will take any default position for the enemy plane.

8)**Friend class**: Declare Radar class as a friend of the Airspace class.

9) **Check whether an enemy plane has entered the airspace zone**: Write a member function for the Radar class that checks whether an enemy plane crosses the country's airspace boundary.

void checkViolation(Airspace &s) …….. (viii)

As you see, **checkViolation** takes an airspace object as the function parameter to determine whether any enemy plane has entered that airspace by comparing its position. If an enemy plane is found to be inside the airspace boundary, **checkViolation** will print the following:

" Airspace violation happened!"

Otherwise, the program prints: "No airspace violation has happened!"

Use the attached **interception_tester.cpp** file for testing purposes. You do not need to check invalid input in this assignment. The attached video demo shows how the program works.

Submission:

Compile and execute your program with g++ or Microsoft compiler.

Place your solution in a zipped file named with your last name followed by the first initial of your first name followed by 2 (ex: YasminS2.zip) and submit the solution via canvas.

Include at the top of your solution (in comments) your name, what compiler(s) you used.

Thus, your zip will contain the following:

- Point3D.h, Point3D.cpp, Airspace.h, Airspace.cpp, Radar.h, and Radar.cpp

Submission deadline is Monday, April 20, 11:59 pm.

This assignment weighs 10% of the course.